

Oriëntatie op AI

Les 7: data science – data-analyse en -visualisatie

2025-2026 Semester 1

HBO-ICT



Agenda

1. Data inladen en conversies
2. Staafdiagram, histogram en boxplot
3. Spreidingsdiagram
4. Beslisboom
5. Gegroepeerde staafdiagram
6. Prestaties



Datavisualisatie

We weten nu hoe we data kunnen samenvatten aan de hand van centrum- en spreidingsmaten.

- Dit kunnen we gebruiken om nieuwe – onbekende - data te verkennen!

Er is nog een manier om je data beter te leren kennen: **datavisualisatie**.



Data inladen

Maar, voordat we met datavisualisatie aan de slag moeten we de data eerst *inladen*.

- We weten hoe we data uit een *database* kunnen inladen vanuit Python.
- Maar data wordt vaak ook bewaard in *Excel* of CSV (comma separated values) bestanden.
 - Vandaag gaan we werken met het bestand `vrijwilligers.csv`, een dataset waar informatie staat over lokaal vrijwilligerswerk.

Intern_results.csv

Deze dataset ziet er als volgt uit:

```
,project,jaar,aantal_vrijwilligers,aantal_uren,percentage_buitenwerk,actief,succes
0,Toekomstmakers,2022,2,30,0.64,True,True
```

- Een naamloze (id)kolom.
- project: de naam van het vrijwilligersproject.
- jaar: het jaar van uitvoering.
- aantal_vrijwilligers: het aantal vrijwilligers dat hielp.
- aantal_uren: het aantal uren dat besteed is.
- percentage_buitenwerk: het percentage aan uren buiten kantoor.
- actief: of het project actief was.
- succes: of het project een succes was.

Data inladen

Om data in te laden vanuit de CSV, kunnen we de csv package in Python gebruiken.

- Deze code slaat de CSV op in een *dictionary* van *lijsten*.
- Het resultaat ziet er zo uit:
 - Welke datatypes zie je?

```
with open('vrijwilligers.csv') as f:
    reader = csv.DictReader(f)
    headers = reader.fieldnames
    data = {}

    for header in headers:
        data[header] = []

    for row in reader:
        for header in headers:
            data[header].append(row[header])
```

```
{'': ['0', ...],
 'project': ['Toekomstmakers', ...],
 'jaar': ['2022', ...],
 'aantal_vrijwilligers': ['2', ...],
 'aantal_uren': ['30', ...],
 'percentage_buitenwerk': ['0.64', ...],
 'actief': ['True', ...],
 'succes': ['True', ...]
}
```

Oefening

Vind het aantal verschillende projecten dat is uitgevoerd.

- Gebruik hiervoor de CSV genaamd vrijwilligers.csv
 - Deze vind je op Github.
- Gebruik onderstaande code om de data in te laden.

```
with open('vrijwilligers.csv') as f:
    reader = csv.DictReader(f)
    headers = reader.fieldnames
    data = {}

    for header in headers:
        data[header] = []

    for row in reader:
        for header in headers:
            data[header].append(row[header])
```

Datatypes en meetniveaus

Voor de meeste berekeningen moeten we eerst de *datatypes* en *meetniveaus* bepalen, en de data converteren.

- Per meetniveau zijn andere bewerkingen mogelijk (zie vorige les).

Niveau	Categorieën	Rangorde	Gelijke afstanden	Absoluut nulpunt	Voorbeeld	Bewerkingen mogelijk
Nominaal	✓	✗	✗	✗	Geslacht, haarkleur	Tellen
Ordinaal	✓	✓	✗	✗	Opleidings niveau	Tellen, sorteren
Interval	✗	✓	✓	✗	Tijd in uren, °C	Optellen/aftrekken, gemiddelde
Ratio	✗	✓	✓	✓	Lengte, gewicht, Kelvin	Alle bewerkingen

Oefening

Laten we de types en meetniveaus bepalen:

Kolom	Type	Meetniveau
Id		
project		
jaar		
aantal_vrijwilligers		
aantal_uren		
percentage_buitenwerk		
actief		
succes		

Datatypes en meetniveaus

We kunnen het inlaadscript uitbreiden met de benodigde conversies:

```
import csv
with open('vrijwilligers.csv') as f:
    reader = csv.DictReader(f)
    headers = reader.fieldnames
    data = {}

    for header in headers:
        data[header] = []
    for row in reader:
        row[''] = int(row[''])
        row['jaar'] = int(row['jaar'])
        row['aantal_vrijwilligers'] = int(row['aantal_vrijwilligers'])
        row['aantal_uren'] = int(row['aantal_uren'])
        row['percentage_buitenwerk'] = float(row['percentage_buitenwerk'])
        row['actief'] = row['actief'] == 'True'
        row['succes'] = row['succes'] == 'True'
    for header in headers:
        data[header].append(row[header])
```

Oefening

Laten we beginnen met berekeningen die we al kennen; bereken de volgende centrummaten:

- Het gemiddelde percentage buitenwerk.
- De mediaan van het aantal vrijwilligers

Oefening

Laten

volgen

- He

- De

```
import csv
with open('vrijwilligers.csv') as f:
    reader = csv.DictReader(f)
    headers = reader.fieldnames
    data = {}

    for header in headers:
        data[header] = []
    for row in reader:
        row[''] = int(row[''])
        row['jaar'] = int(row['jaar'])
        row['aantal_vrijwilligers'] = int(row['aantal_vrijwilligers'])
        row['aantal_uren'] = int(row['aantal_uren'])
        row['percentage_buitenwerk'] = float(row['percentage_buitenwerk'])
        row['actief'] = row['actief'] == 'True'
        row['succes'] = row['succes'] == 'True'
    for header in headers:
        data[header].append(row[header])
```

en de

- Gebruik het (aangevulde) inlaad- en conversiescript.

Modus en frequenties

Om de modus te berekenen, is een stuk meer code nodig (zie ook opdracht AI-02).

Gelukkig kunnen we hiervoor ook de module `collections` gebruiken!

```
from collections import Counter  
print(Counter(data['aantal_vrijwilligers']))
```

```
Counter({1: 35, 2: 23, 3: 10, 4: 9, 7: 8, 0: 6, 6: 4, 5: 3, 8: 1})
```

- Wat is de modus van `aantal_vrijwilligers`?
- Wat zie je nog meer?

Modus en staafdiagram

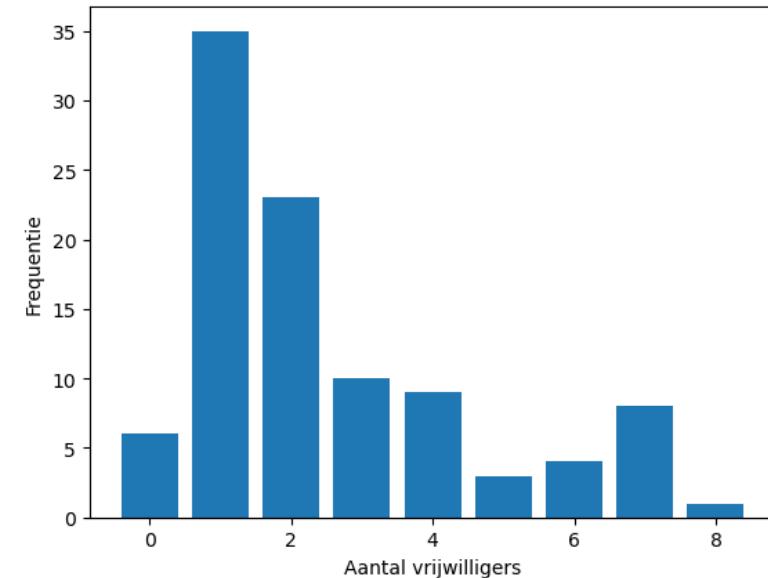
De frequenties kunnen we ook weergeven in een **staafdiagram**.

- Hier geeft de *staafhoogte* aan hoeveel een waarde voorkomt.

```
from collections import Counter
frequencies = Counter(data['aantal_vrijwilligers'])

Counter({1: 35, 2: 23, 3: 10, 4: 9, 7: 8, 0: 6, 6: 4, 5: 3, 8: 1})
```

```
import matplotlib.pyplot as plt
plt.bar(frequencies.keys(), frequencies.values())
plt.xlabel('Aantal vrijwilligers')
plt.ylabel('Frequentie')
plt.show()
```



Uit een visualisatie kun je vaak makkelijker *patronen herkennen!*

- Wat valt nu allemaal op?

Oefening

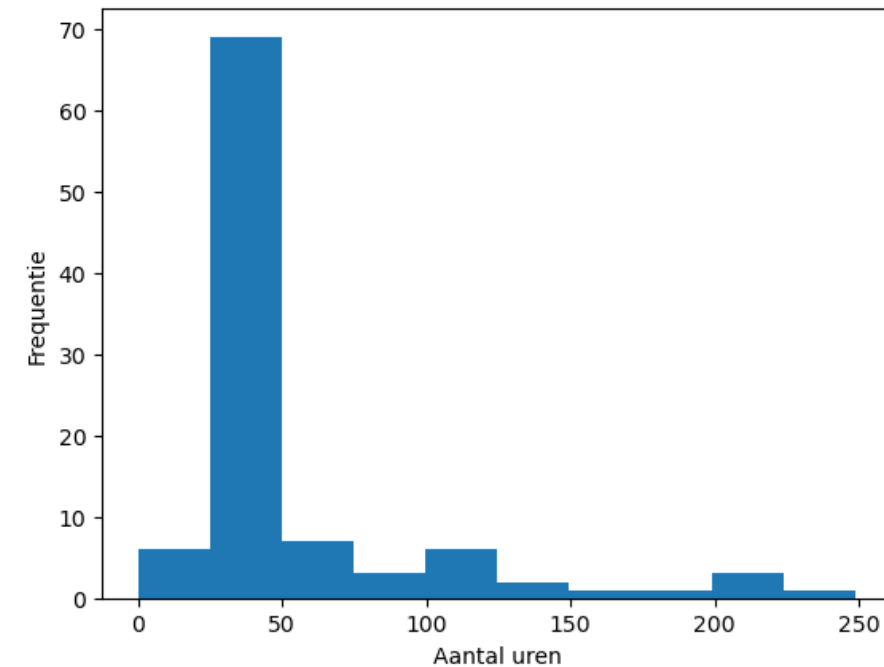
Maak een staafdiagram van aantal_uren.

Histogram

Een visualisatie die dit probleem oplost is de **histogram**.

- Deze groepeerde de frequenties in intervallen (ook wel *bins*)
- Hier worden standaard 10 bins getoond.

```
import matplotlib.pyplot as plt
plt.xlabel('Aantal uren')
plt.ylabel('Frequentie')
plt.hist(data['aantal_uren'])
```



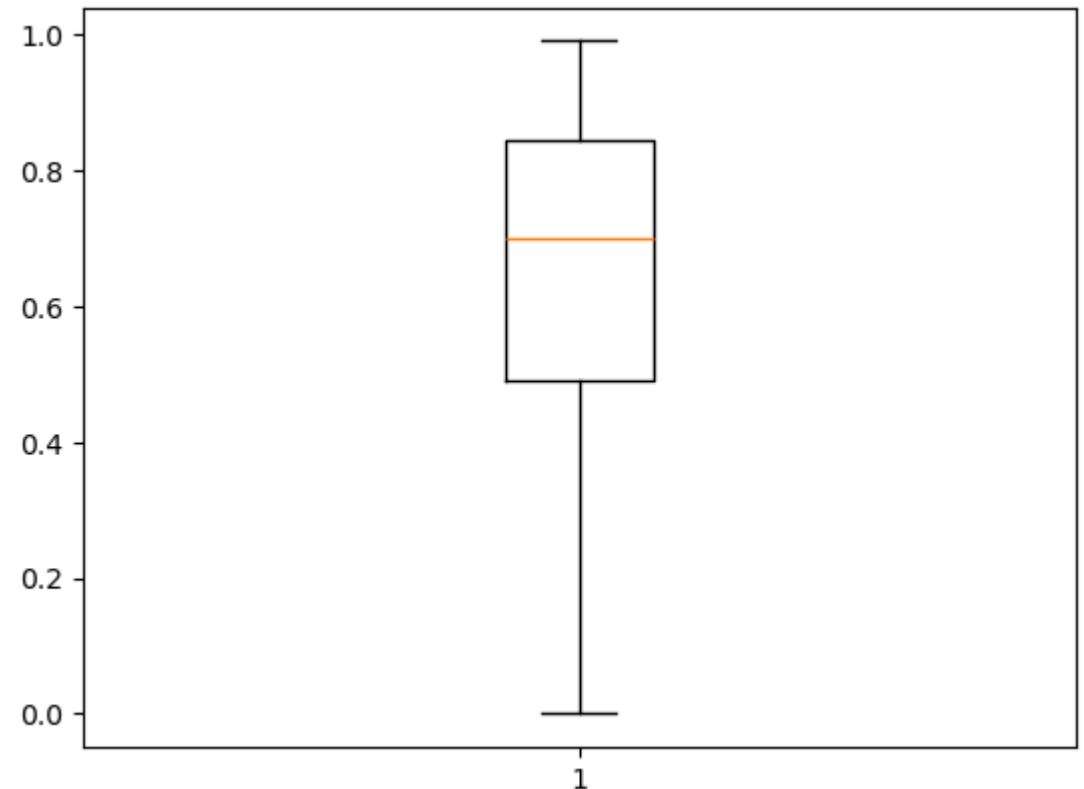
- De modus is niet meer te achterhalen, waarom?

Boxplot

Een andere visualisatie om de verdeling van een variabele weer te geven kennen we al: de **boxplot**.

```
import matplotlib.pyplot as plt
plt.boxplot(data['percentage_buitenwerk'])
plt.show()
```

Wat zie je hier?



Oefening

Maak een boxplot van aantal_vrijwilligers.

Univariate vs bivariate visualisaties

De boxplot en (in de getoonde voorbeelden) de staafdiagram zijn voorbeelden van **univariate** visualisaties.

- Hier wordt *één variabele* weergegeven.

Om meer te weten te komen over relaties tussen *twee variabelen* hebben we een **bivariate** visualisatie nodig.

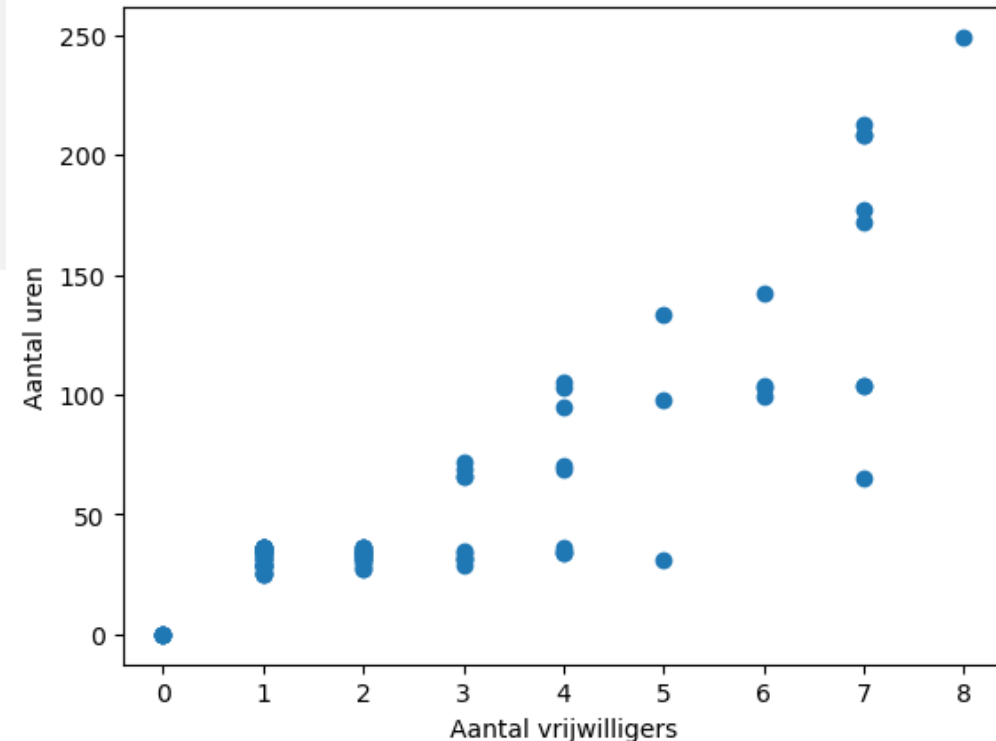
Spreidingsdiagram

Een voorbeeld hiervan is de **spreidingsdiagram** (ook wel *scatter plot*).

- Deze toont een punt op de waarden van de twee variabelen.

```
import matplotlib.pyplot as plt
plt.scatter(data['aantal_vrijwilligers'], data['aantal_uren'])
plt.xlabel('Aantal vrijwilligers')
plt.ylabel('Aantal uren')
plt.show()
```

- Hoe verhouden aantal_vrijwilligers en aantal_uren zich tot elkaar?



Oefening

Maak een spreidingsdiagram van jaar ten opzichte van het percentage_buitenwerk.

Data-analyse

We hebben nu verschillende visualisaties gemaakt, en de data op verschillende manieren bekeken.

Maar data-analyse dient altijd een **doel**, en binnen data science is een van de belangrijkste doelen om de toekomst te voorspellen.

We gaan vandaag proberen te voorspellen wanneer een vrijwilligersproject een **succes** wordt.

Succes voorspellen

Om de toekomst te voorspellen is gelukkig geen magie nodig. Alle kennis die we nodig hebben zit verstopt *in de data*.

Kolom	Type	Meetniveau
Id	int	Ordinaal
project	string	Nominaal
jaar	int	Interval
aantal_vrijwilligers	int	Ratio
aantal_uren	int	Ratio
percentage_buitenwerk	float	Ratio
actief	bool	Nominaal
succes	bool	Ordinaal

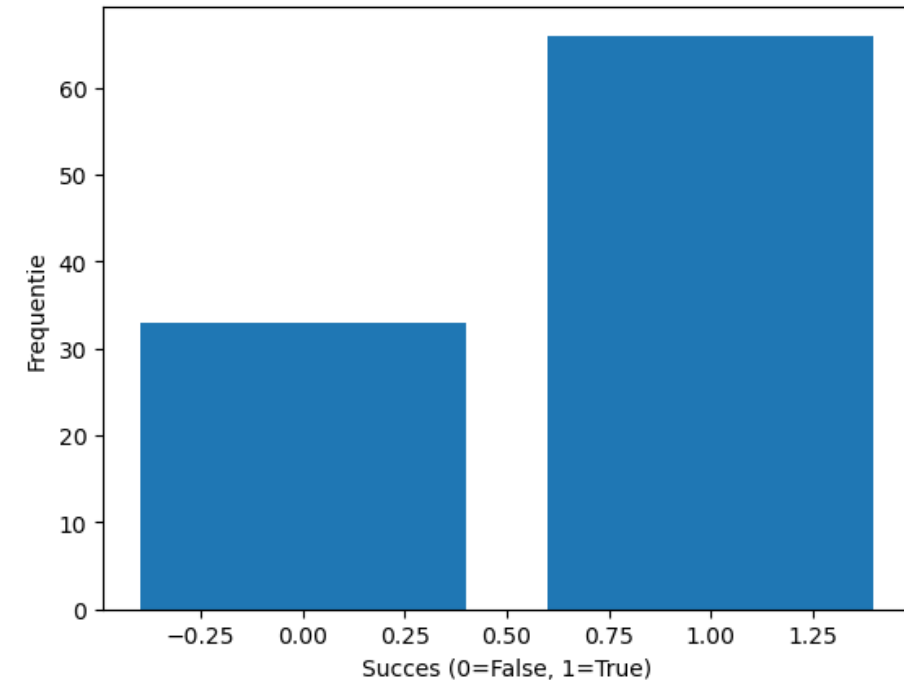
- Dus welke kolom gaan we voorspellen?

Succes voorspellen

We gaan inderdaad de **targetvariabele** succes proberen te voorspellen.
Laten we die kolom eens gaan bekijken!

```
from collections import Counter  
frequencies = Counter(data['succes'])
```

```
import matplotlib.pyplot as plt  
plt.bar(frequencies.keys(),  
frequencies.values())  
plt.xlabel('Succes (0=False, 1=True)')  
plt.ylabel('Frequentie')  
plt.show()
```



- Wat zien we hier?
- Wat gaan we dus proberen voorspellen?

Succes voorspellen

Dit gaan we voorspellen door gebruik te maken van de overige kolommen, ook wel de **featurevariabelen**.

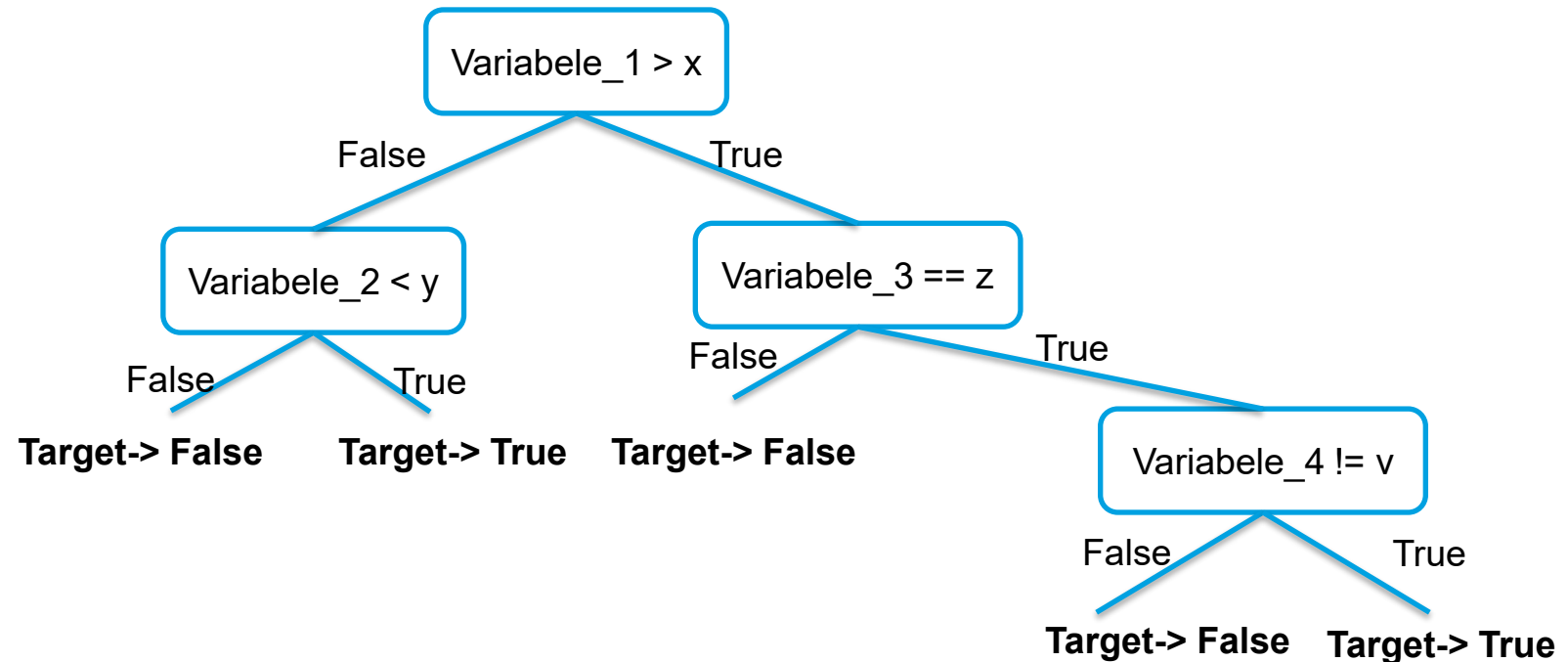
Oftewel, de variabelen:

- jaar
- aantal_vrijwilligers
- aantal_uren
- percentage_buitenwerk
- actief
- (de id-kolom kunnen we negeren)

Beslisboom

Een hele intuïtieve manier om een variabele te voorspellen is door gebruik te maken van een **beslisboom**.

- Dit eenvoudige model koppelt in feite een aantal *ifelse statements* aan elkaar.



Beslisboom

We kunnen zo'n beslisboom gebruiken om de targetwaarde te voorspellen van data met projecten.

Dit doen we door de boom aan de hand van zo'n project van boven naar beneden af te lopen.

- Kom in een *bladknoop*? Dan vind je de voorspelling!

Een beslisboom is een veel gebruikt *machine learning algoritme*.

- Dit algoritme vindt automatisch de optimale beslisboom.
- De rest van de les gaan we zelf zo'n boom maken!

Beslisboom

De truc is om de ifelse beslissingen zo te maken dat na een splitsing elke vertakking zo veel mogelijk *uit één targetwaarde* bestaat.

- Dus met succes==True óf success==False.
- Het makkelijkste is om een featurevariabele hierop te **splitsen**.
- Voor de variabele actief zouden we dit zo kunnen doen:

```
successful = []
unsuccessful = []
for i in range(len(data['succes'])):
    if data['succes'][i]:
        successful.append(data['actief'][i])
    else:
        unsuccessful.append(data['actief'][i])
unsuccessful_freqs = Counter(unsuccessful)
successful_freqs = Counter(successful)
```

Gegroepeerde staafdiagram

Een visualisatie waarmee je de succesvolle en de onsuccesvolle projecten in één overzicht kan zien, is de **gegroepeerde staafdiagram**.

- Python heeft vele packages om dit mooi weer te geven.
- In matplotlib is dit het makkelijkste:

```
plt.bar(unsuccesful_freqs.keys(), unsuccesful_freqs.values(), width=0.5, label='Onsuccesvol')
# plt.show()
plt.bar(successful_freqs.keys(), successful_freqs.values(), width=0.5, align='edge', label='Succesvol')
plt.xlabel('Actief (0=False, 1=True)')
plt.ylabel('Frequentie')
plt.legend()
plt.show()
```

1. We maken de staven van beide plots minder breed (bv: 0.5).

2. We plotten nog maar één keer, aan het einde.

3. We alignen de tweede plot rechts tegen de rand (edge).

5. We tonen een legenda.

4. We geven elke groep een label.

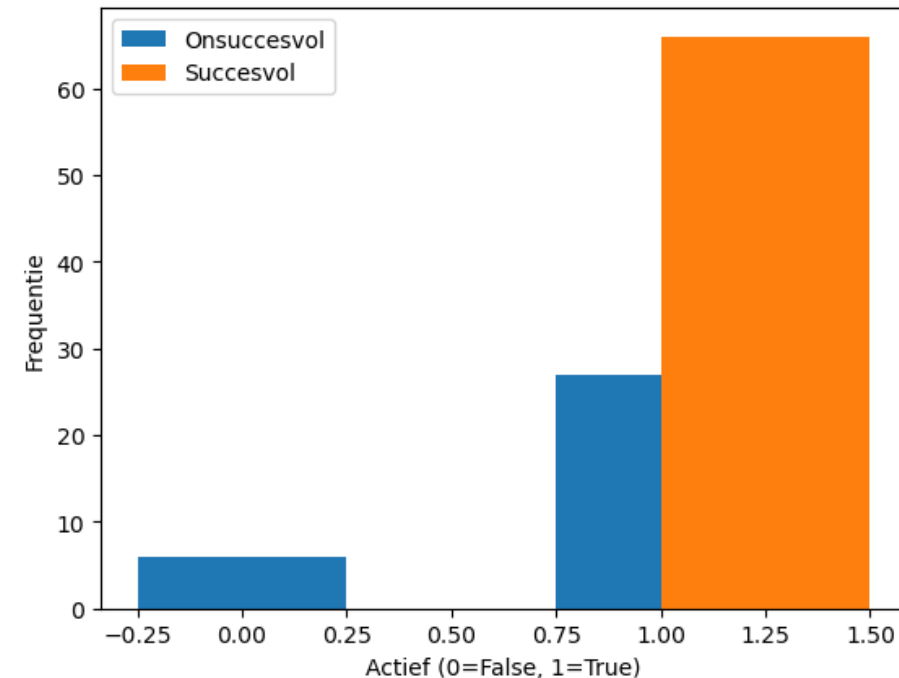


Gegroepeerde staafdiagram

Het resultaat: één overzichtelijke grafiek waar je in een enkele oogopslag ziet waar je het beste kunt splitsen.

We zien:

- Inactieve projecten zijn altijd onsuccesvol.
- Actieve projecten zijn vaker succesvol.



Wat is dus de meest logische voorspelling?

- Inactief->onsuccesvol, actief-> succesvol

Beslisboom prestaties

Maar hoe goed voorspelt deze eenvoudige beslisboom nou?

- Dat is gewoon een kwestie van tellen!

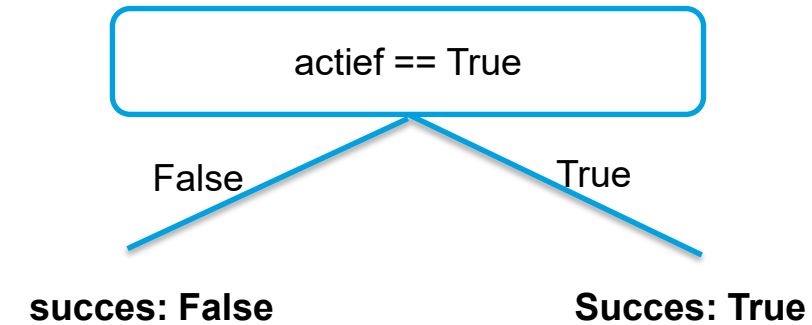
	active	not active
succes	66	0
not success	27	6
Totaal	93	6

- Bij actieve projecten zijn 66 van de 93 (71%) succesvol.
- Bij inactieve projecten zijn 6 van de 6 (100%) niet succesvol.
- Het model voorspelt dus 72 van de 99 keer (72,7%) goed.
 - Niet geweldig dus, maar beter dan een muntje opgooien.

Beslisboom

- Deze beslisboom begint dus als volgt:

```
data['succes_prediction'] = []  
for i in range(len(data['succes'])):  
    if data['actief'][i]:  
        data['succes_prediction'].append(True)  
    else:  
        data['succes_prediction'].append(False)
```



- En de prestaties van de boom vinden we zo:

```
correct = 0  
for i in range(len(data['succes'])):  
    if data['succes'][i] == data['succes_prediction'][i]:  
        correct += 1  
print(f"{correct} / 99 = {round(correct/99*100, 1)}%" # 72 / 99 = 72.7%
```


Oefening

Doe dit nou eens voor `aantal_vrijwilligers`.

- Hoe kunnen we het beste splitsen?
- Hoe goed voorspelt dit model?

Oefening

Ga op zoek naar de beste splitsing.

- Tip: Speel met de breedte om alle variabelen in een gegroepeerde staafdiagram inzichtelijk te krijgen.

- Gebruik deze code en pas aan

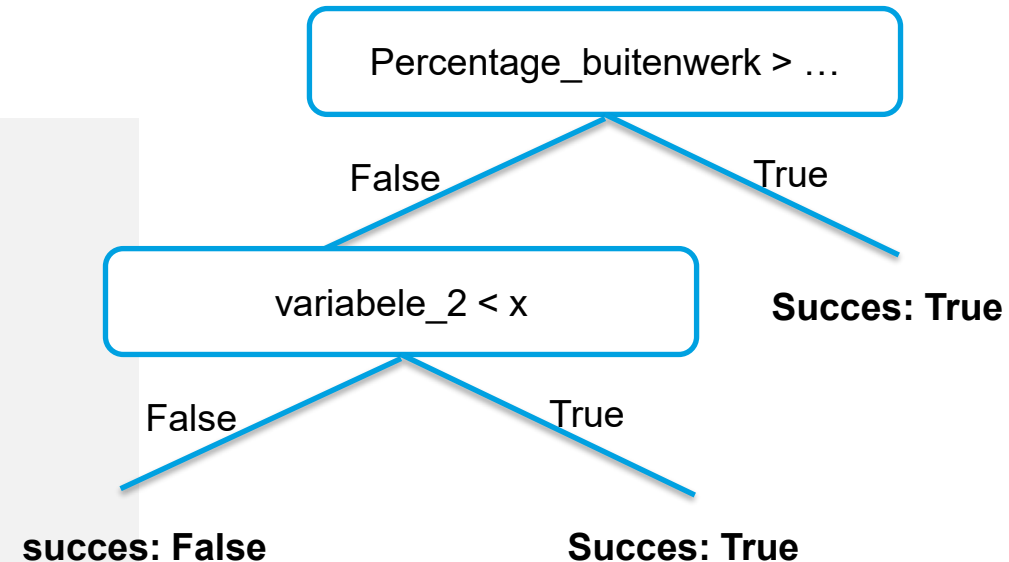
```
data['succes_prediction'] = []
for i in range(len(data['succes'])):
    if data['actief'][i]:
        data['succes_prediction'].append(True)
    else:
        data['succes_prediction'].append(False)
```

```
correct = 0
for i in range(len(data['succes'])):
    if data['succes'][i] == data['succes_prediction'][i]:
        correct += 1
print(f"{correct} / 99 = {round(correct/99*100, 1)}%" # 72 / 99 = 72.7%
```

Meer knopen

Door meer beslisknopen toe te voegen kunnen we waarschijnlijk nog wel een betere voorspelling doen

```
data['succes_prediction'] = []
for i in range(len(data['succes'])):
    if data['percentage_buitenwerk'][i] > ...:
        if data['variabele_2'][i] < x:
            data['succes_prediction'].append(False)
        else:
            data['succes_prediction'].append(True)
    else:
        data['succes_prediction'].append(False)
```



Overfitten

Het risico van meer knopen toevoegen is dat het model te nauw aansluit op de data, deze is dan **overfit**.

- Overfitte modellen presteren slecht op nieuwe, ongeziene, data.
 - Dit zouden in ons geval toekomstige projecten zijn.
- Hoe je hier mee om moet gaan is voorbij de inhoud van deze les.

Huiswerk

- Oefening AI7.1: Beslisboom (zie Canvas)
 - Voeg op zijn minst één extra knoop toe aan de beslisboom die de prestaties verbetert - hoger dan 84,8%.
- Ga verder met Opdracht AI2: Data science (zie Canvas)